RemSolution — Roadmap & Plan of Work (Multi-tenant SaaS + Marketplace)

July 3, 2026 · Based on the current project state (~30% of use cases implemented)

The Clean Architecture foundation, data model, and infrastructure are solid. Target product: a multi-tenant SaaS where rental agencies (societies) subscribe with limits on cars/clients, plus a public Airbnb-style marketplace where end clients pick a country and see available cars from nearby agencies. Tenancy model: **single database, shared schema, AgencyId on every tenant table**, enforced by EF Core global query filters — decided now because retrofitting AgencyId after the renting core is built is the expensive mistake. Plan: fix the hygiene debt (~2–3 days), lay the tenancy foundation (~3–4 days), then vertical slices in dependency order: **Client** → **Renting** → **Reservation/Payment** → **Expense** → **Hardening** → **Marketplace**. Total estimate: **9–10 weeks**.

Performance note: at realistic scale (thousands of agencies, ~100k cars, millions of renting rows/year) this is a small OLTP workload for SQL Server. The real risks are indexes that don't lead with AgencyId and a naive availability query — both are addressed as explicit tasks below.

If scope must be cut: cut Phase 4, and defer Phase 6 (marketplace) to a second release. Reservation is no longer cuttable — marketplace bookings enter as Reservations. Phases 0–3 are the non-negotiable spine.

Phase 0 — Hygiene & unresolved decisions (2–3 days)

Do this before any feature work. Every item gets more expensive the more code exists.

#	Task	What to do	Effort
0.1	**Retarget to .NET 10 (LTS)**	.NET 9 is STS and out of support since May 2026; the SDK is already 10.0.101. Change **Directory.Build.props** to net10.0, bump packages in **Directory.Packages.props** (EF Core, ASP.NET, MediatR, etc.), delete stale obj/net8.0 artifacts, rebuild, re-run the 10 unit tests.	0.5 d
0.2	**Squash the EF migration (non-negotiable now)**	InitialCreate still contains Todo tables and PendingModelChangesWarning is suppressed — the migration no longer describes the model, and AgencyId is about to be added to every tenant table on top of it. Assuming no production DB exists: delete the Migrations folder, drop LocalDB, regenerate InitialCreate, remove the warning suppression. If a deployed DB exists anywhere, write a corrective migration that drops the Todo tables instead — do NOT squash.	0.5 d
0.3	**Resolve Domain.User vs ApplicationUser**	Delete **Domain.User**. Keep ApplicationUser in Infrastructure as the single identity source of truth; reference users from the domain by string UserId only (the template's IUser abstraction already exists). Client stays a separate domain entity — a client is not necessarily a login. Must be settled before Renting, which references clients and audit users.	0.5 d
0.4	**Externalize the connection string**	Uncomment the configuration line in Infrastructure/DependencyInjection.cs. Local: user secrets. Azure: Key Vault / environment variables (plumbing already present). Remove the hardcoded value from source.	0.1 d
0.5	**Delete template-leftover domain events**	Remove CarCompletedEvent (raised on create — wrong), the never-raised Deleted events, and the log-only handlers. Also delete the leftover PriorityLevel enum. Reintroduce events only when Renting gives a real consumer (e.g. RentingCompletedEvent → write RentingHistory).	0.3 d

#	Task	What to do	Effort
0.6	**Drop Bootstrap, keep Angular Material**	Two UI systems are loaded. Material already owns the tables/paginator — remove bootstrap 5 and ngx-bootstrap from package.json and angular.json, fix any components that used them. Do this before building Client/Renting screens.	0.5 d
0.7	**Complete the small CRUD gaps**	Add Update commands + endpoints for Brand, ModelCar, Country (copy the Car update slice). Country: prefer seeding the reference data and skipping the UI entirely unless the business edits countries.	0.5 d

Phase 0.5 — Multi-tenancy & subscription foundation (3–4 days)

Single database, shared schema. Everything after this phase is tenant-scoped for free; skipping it means retrofitting AgencyId across every slice later.

#	Task	What to do	Effort
T.1	**Agency (society) root entity**	Agency: name, contact info, CountryId, and a **geography Location column** (needed for 'nearby' in Phase 6 — add it now so the migration is done once). Enable NetTopologySuite: UseSqlServer(x => x.UseNetTopologySuite()). CRUD + endpoints (platform-admin only).	1 d
T.2	**AgencyId on every tenant table**	Add AgencyId FK to Car, Client, Renting, Reservation, Payment, Expense, ExtraService. Indexing rule from day one: **tenant-scoped indexes lead with AgencyId** (e.g. IX Car(AgencyId, ...)), otherwise every agency dashboard query scans all tenants' data.	0.5 d
T.3	**Tenant resolution + enforcement**	ITenantProvider resolved per-request from the JWT AgencyId claim. EF **global query filters** on every tenant entity (HasQueryFilter(e => e.AgencyId == tenant.AgencyId)) so no handler can leak cross-tenant data by forgetting a WHERE. A SaveChanges interceptor auto-stamps AgencyId on insert (same pattern as AuditableEntityInterceptor). Cross-tenant reads via IgnoreQueryFilters() are allowed in exactly ONE place: the Phase 6 MarketplaceSearch feature folder — never in agency-facing handlers.	1 d
T.4	**Subscription plans + limit enforcement**	SubscriptionPlan (MaxCars, MaxClients, price) and AgencySubscription (agency, plan, period, status). Enforce limits in CreateCar/CreateClient handlers: count + insert inside ONE transaction under sp_getapplock keyed on AgencyId — a bare SELECT COUNT is a race (two concurrent creates both pass the check). Expired subscription → block writes, keep reads. Payment/billing provider integration is out of scope for now; status is managed manually by the platform admin.	1 d
T.5	**Roles: platform admin vs agency admin vs agency staff**	Extend Identity: PlatformAdministrator (manages agencies/plans), AgencyAdministrator, AgencyStaff. Agency users carry the AgencyId claim; platform admins do not (and bypass tenant filters via a dedicated, audited path). Seed one platform admin.	0.5 d

Phase 1 — Client vertical slice (1 week)

First full slice on the new foundation — it becomes the template for every remaining feature. Get validation, DTO, and endpoint patterns right here.

#	Task	What to do	Effort
1.1	**Commands / queries**	CreateClient, UpdateClient, DeleteClient, GetClientById, GetClientsWithPagination — same vertical-slice layout as Car (Commands / Queries / DTOs / Validators per feature folder). Mapster ProjectToType for reads. Tenant scoping comes free from the Phase 0.5 filters. Add a nullable **MarketplaceUserId** on Client now: in Phase 6 a self-registered marketplace user gets one global account and a linked Client row per agency on first booking (Airbnb model) — reserving the column avoids another migration later.	2 d
1.2	**Validation**	FluentValidation rules for CIN, Passeport, and Driving Licence blocks (number format, issue date not in the future, expiry logic where applicable), required name/birth fields, valid Country FKs.	0.5 d
1.3	**Document image storage**	Decide now: store paths/URIs in SQL + files in blob storage (or wwwroot/uploads in dev) — NOT varbinary in the Client row. Add an IFileStorage abstraction in Application, implement in Infrastructure.	1 d
1.4	**EF configuration review**	Client has multiple FKs to Country. SQL Server rejects multiple cascade paths — set DeleteBehavior.Restrict/NoAction explicitly on these relationships and verify the regenerated migration.	0.3 d
1.5	**Angular Client CRUD**	List with Material table + paginator + search, create/edit form (incl. image upload), delete with confirm. Regenerate the NSwag client after the endpoints land.	1.5 d

Phase 2 — Renting core (2 weeks)

The actual product. Includes the one genuinely hard concurrency problem in the system.

#	Task	What to do	Effort
2.1	**RentingState decision**	Done/InProgress/NotYet is derived from dates + return event. Either compute it in queries, or make it an explicit state machine with guarded transitions (NotYet → InProgress → Done). Decide one — hand-updated stored state is where inconsistent data comes from. Recommended: explicit state machine, since the return event carries data (end mileage).	0.5 d
2.2	**CreateRentingCommand + availability check**	Create a renting for car + primary/secondary client with dates, start mileage, price, extra services. The availability check MUST be race-safe: two concurrent requests can both see the car as free. Use sp_getapplock keyed on CarId inside a single transaction around the overlap check + insert (serializes per-car only). Overlap predicate: newStart < existingEnd AND newEnd > existingStart — and decide explicitly whether same-day turnaround counts as an overlap.	3 d
2.3	**Return / close flow**	CompleteRentingCommand: capture end mileage (validate ≥ start mileage), transition state to Done, compute final price incl. extra services, write a RentingHistory snapshot on the transition.	2 d
2.4	**ExtraServices on a renting**	Attach/detach ExtraService lines; CRUD for ExtraServicesType (reference data). Include in price calculation.	1 d

#	Task	What to do	Effort
2.5	**Queries + endpoints + availability indexes**	GetRentingById (full detail: car, clients, services, history), paginated list with filters (state, car, client, date range), car-availability query for the UI calendar/picker. Create the indexes the overlap checks and the future marketplace search depend on: **IX Renting(CarId, StartDate, EndDate) INCLUDE (State)** and the mirror on Reservation — these keep the NOT EXISTS anti-joins fast into the millions of rows.	1.5 d
2.6	**Functional tests (as you build, not after)**	The Testcontainers/Respawn scaffolding is empty — use it here. Cover: overlap detection (boundary dates), concurrent create (two parallel requests, one must fail), state transitions, mileage validation. This is exactly the code that regresses silently.	2 d
2.7	**Angular Renting UI**	Renting list + detail, create wizard (pick car → check availability → pick client(s) → dates/price → services), return dialog. Regenerate NSwag client.	2 d

Phase 3 — Reservation + Payment (1–1.5 weeks)

#	Task	What to do	Effort
3.1	**Reservation CRUD**	Create/update/cancel reservations (client, car, dates, price, payed price). Reservations participate in the same availability/overlap check as rentings — a reserved car is not free. This matters doubly for Phase 6: marketplace bookings land as Reservations, so the overlap predicate must cover both tables from the start.	1.5 d
3.2	**Convert reservation → renting**	ConvertReservationToRentingCommand: creates the renting from the reservation under the same per-car lock, carries over dates/price/advance payment, links Reservation ↔ Renting. This is where that FK earns its keep.	1 d
3.3	**Payment recording**	Record payments against a client and optionally a renting/reservation. Money as decimal(18,2) or a Money value object — verify EF configurations set precision explicitly instead of defaulting. Define the invariant: can PayedPrice exceed Price? Enforce in the validator either way. Add a client balance / outstanding-amount query.	1.5 d
3.4	**Angular UI**	Reservation calendar/list + convert action, payment entry form, client account view showing payments vs due.	1.5 d

Phase 4 — Expense & maintenance notifications (1 week)

#	Task	What to do	Effort
4.1	**Expense + ExpenseType CRUD**	Per-car expenses with amount, date, mileage; ExpenseType carries the mileage / month notification thresholds.	1.5 d
4.2	**'Due' dashboard query**	Simplest correct version: a query surfacing cars whose last expense of a type exceeds its mileage or month threshold, shown on a dashboard. Do NOT build a background scheduler / push notifications unless actually requested — add Hangfire/hosted service later if needed.	1.5 d
4.3	**Angular UI**	Expense entry per car, dashboard widget listing overdue maintenance.	1 d

Phase 5 — Hardening (ongoing, ~1 week focused)

#	Task	What to do	Effort
5.1	**Backfill unit tests**	Validators, price calculation, state-machine guards. The domain/application test projects are empty scaffolding.	2 d
5.2	**One end-to-end acceptance path**	SpecFlow + Playwright happy path: create client → rent car → return → record payment. One reliable E2E beats twenty flaky ones.	1.5 d
5.3	**Authorization pass**	Everything is currently RequireAuthorization() with a single Administrator policy. Wire the Phase 0.5 roles into policies: platform-admin endpoints (agencies, plans), agency-admin (delete/purge, financials), agency-staff (day-to-day). Add a functional test proving **tenant isolation**: a user from agency A requesting agency B's renting by Id gets 404, not data.	1 d
5.4	**Seed data + demo dataset**	Extend the seeder: countries, brands/models, a few cars, sample clients — makes every demo and test cheaper.	0.5 d

Phase 6 — Public marketplace (2–3 weeks)

The Airbnb-style layer: a client picks a country and sees available cars from nearby agencies. Depends on Phases 0.5–3 — you cannot sell a marketplace over a back office that cannot complete a rental.

#	Task	What to do	Effort
6.1	**Marketplace user self-registration**	Public Identity registration/login flow, separate from agency staff. One global MarketplaceUser account; a linked Client row is created inside the target agency on first booking (fills the MarketplaceUserId column reserved in task 1.1). No AgencyId claim on these users.	2 d
6.2	**Cross-tenant availability search**	The one query to design carefully — read-only, anonymous-accessible, and the ONLY place that uses IgnoreQueryFilters(). Shape: filter agencies by CountryId + Location.STDistance(@point) < @radius (spatial index on Agency.Location), then NOT EXISTS anti-joins against Renting and Reservation with the overlap predicate (start < existingEnd AND end > existingStart), ordered by distance. Rides on the indexes from task 2.5. Project a lean search DTO (car, agency, distance, price) — no Includes.	3 d
6.3	**Marketplace booking flow**	Public booking creates a Reservation in the target agency under the same per-car sp_getapplock as back-office bookings — one lock discipline for both entry points, or the race comes back through the side door. Agency sees incoming marketplace reservations in its normal Reservation list and converts to Renting via task 3.2. Online payment capture is a later increment; v1 records the reservation with payment on pickup.	3 d
6.4	**Public Angular frontend**	Country picker → map/list of nearby agencies with available cars for the chosen dates → car detail → book. Separate routing area (or separate app) from the agency back office; anonymous until booking. Regenerate NSwag client.	4 d
6.5	**Search-path load sanity check**	Before launch, run the availability search against a seeded dataset at target scale (e.g. 2,000 agencies / 100k cars / 1M+ renting rows) and read the execution plan: spatial index seek on Agency, index seeks (not scans) on the Renting/Reservation anti-joins. Only if anonymous traffic later outgrows this do you add caching or a denormalized search read model — an additive query-side change under CQRS, not a rewrite. Do not build it preemptively.	1 d

Timeline summary

Phase	Scope	Duration	Cuttable?
0	Hygiene & decisions	2–3 days	No
0.5	Multi-tenancy & subscriptions	3–4 days	No — retrofit is the expensive mistake
1	Client slice	1 week	No
2	Renting core	2 weeks	No
3	Reservation + Payment	1–1.5 weeks	No (Phase 6 books via Reservation)
4	Expense & notifications	1 week	Yes

Phase	Scope	Duration	Cuttable?
5	Hardening	1 week	Partially — keep 5.3 (tenant isolation)
6	Public marketplace	2–3 weeks	Deferable, not cuttable — it is the product goal

Definition of done for the project: a marketplace client can pick a country, see available cars from nearby agencies for their dates, and book; the agency completes the rent → return → pay flow — with functional tests covering overlap logic, state transitions, subscription limits, and tenant isolation.